

TP7 - Pipeline Big Data de supervision des accès avec Kafka, Spark et MongoDB/HDFS

Objectif général	Construire un pipeline de surveillance des accès capable d'ingérer des journaux, de calculer un score de risque et de générer des alertes de sécurité.
Technologies	Kafka ou simulation producteur/consommateur, Spark Structured Streaming ou PySpark batch, MongoDB, HDFS local ou dossier data lake.
Livrable final	Pipeline opérationnel + dashboard simple + rapport d'incidents + règles d'alerte justifiées.

1. Scénario professionnel

Une organisation collecte des milliers de journaux d'accès issus d'applications, bases de données et services Big Data. Le responsable sécurité veut identifier les comportements anormaux: trop d'échecs, exports massifs, accès nocturnes, accès depuis IP inhabituelles, contournement du MFA et accès aux données sensibles.

Architecture attendue

Applications -> Kafka topic access_logs -> Spark Streaming / PySpark -> enrichissement utilisateurs + ressources -> calcul risk_score -> MongoDB alerts + HDFS/Data Lake -> tableau de bord et rapport.

2. Données et environnement

- datasets/access_logs.csv: journaux d'accès simulés.
- datasets/users.csv et resources.csv: enrichissement des événements.
- notebooks/TP2_Kafka_Spark_Audit_Logs.ipynb: version notebook.
- config/docker-compose.yml: démarrage optionnel de Kafka, MongoDB et Spark.

3. Déroulement détaillé du TP

Temps	Activité	Production attendue	Points de vigilance
0:00-0:20	Analyser le schéma des logs et définir les événements suspects.	Liste de 5 règles d'alerte.	Ne pas confondre échec isolé et comportement anormal.
0:20-0:55	Créer le producteur Kafka ou simuler le flux depuis CSV.	Événements publiés dans access_logs.	Respecter un format JSON stable.
0:55-1:40	Créer le traitement Spark: parse JSON, enrichissement avec users/resources.	DataFrame enrichi avec rôle, sensibilité, action.	Contrôler les types de données et timestamps.
1:40-2:25	Calculer risk_score: échec, nuit, export, IP externe, MFA absent, ressource sensible.	Score et niveau faible/moyen/élevé.	Chaque règle doit être expliquée.
2:25-3:05	Écrire les alertes dans MongoDB ou un dossier HDFS local.	Collection alerts ou fichiers parquet/csv.	Ne stocker que les champs nécessaires.
3:05-3:25	Créer un tableau de bord simple dans notebook.	Top utilisateurs à risque, chronologie, types d'incidents.	Relier au besoin de preuve.
3:25-3:40	Rédiger une mini réponse à incident.	Plan de remédiation en 5 actions.	Inclure actions immédiates et préventives.
3:40-4:040	Rédiger présentation PPTX		

4. Règles de risque proposées

Règle	Score	Justification sécurité	Action SOC
Authentification échouée	+2	Peut indiquer un mot de passe deviné ou volé	Surveiller répétitions
Accès hors heures ouvrées	+2	Contexte inhabituel	Demander justification
Export de données	+3	Risque d'exfiltration	Valider autorisation
IP externe ou rare	+3	Origine suspecte	Bloquer temporairement
MFA non validé	+2	Contournement d'un contrôle fort	Forcer réenrôlement MFA
Ressource sensible	+2	Impact élevé en cas d'abus	Revue manuelle

5. Sorties attendues

- Un fichier ou une collection alerts contenant les événements dont risk_score >= 6.
- Un tableau des 10 utilisateurs les plus risqués.
- Un graphique simple des alertes par heure ou par action.
- Un paragraphe d'interprétation: quelles mesures de contrôle d'accès doivent être renforcées ?
- Dashboard avec Flask et hébergé en ligne sur horeku ou tout autre serveur gratuit.

6. Questions de réflexion

- Pourquoi Kafka est-il utile lorsque les journaux arrivent en continu ?
- Pourquoi Spark est-il adapté à la corrélation de grands volumes de logs ?
- Quelles limites présente un scoring par règles ?
- Quels champs faut-il éviter de conserver dans les logs pour réduire le risque de fuite ?